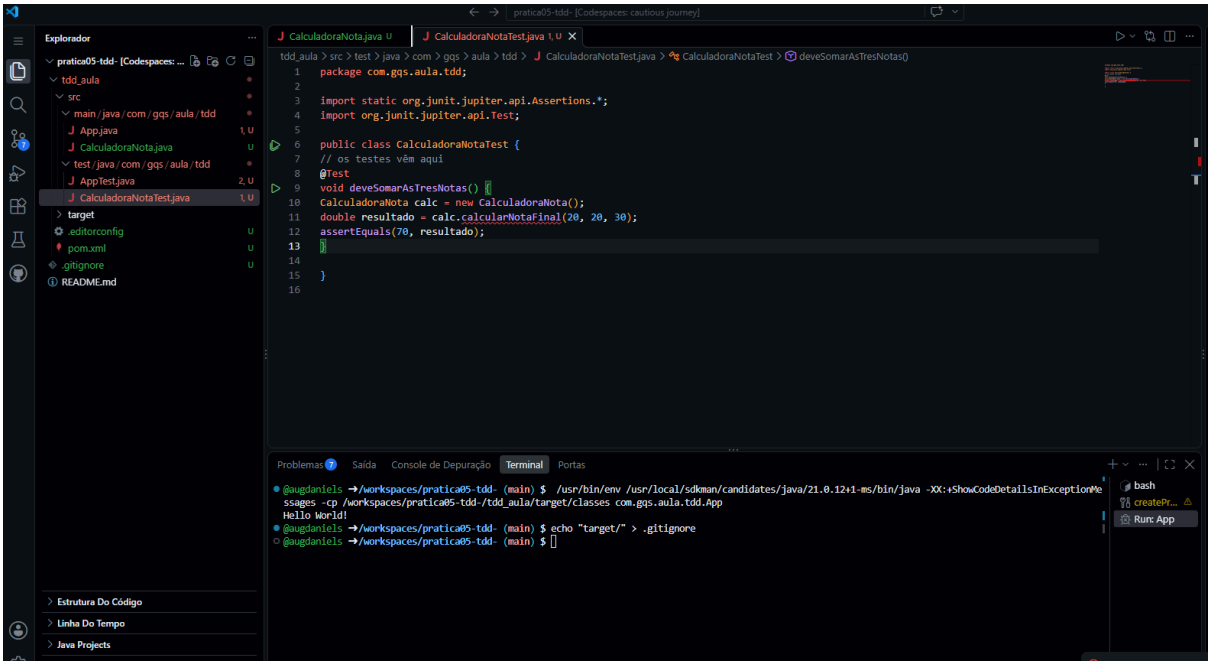
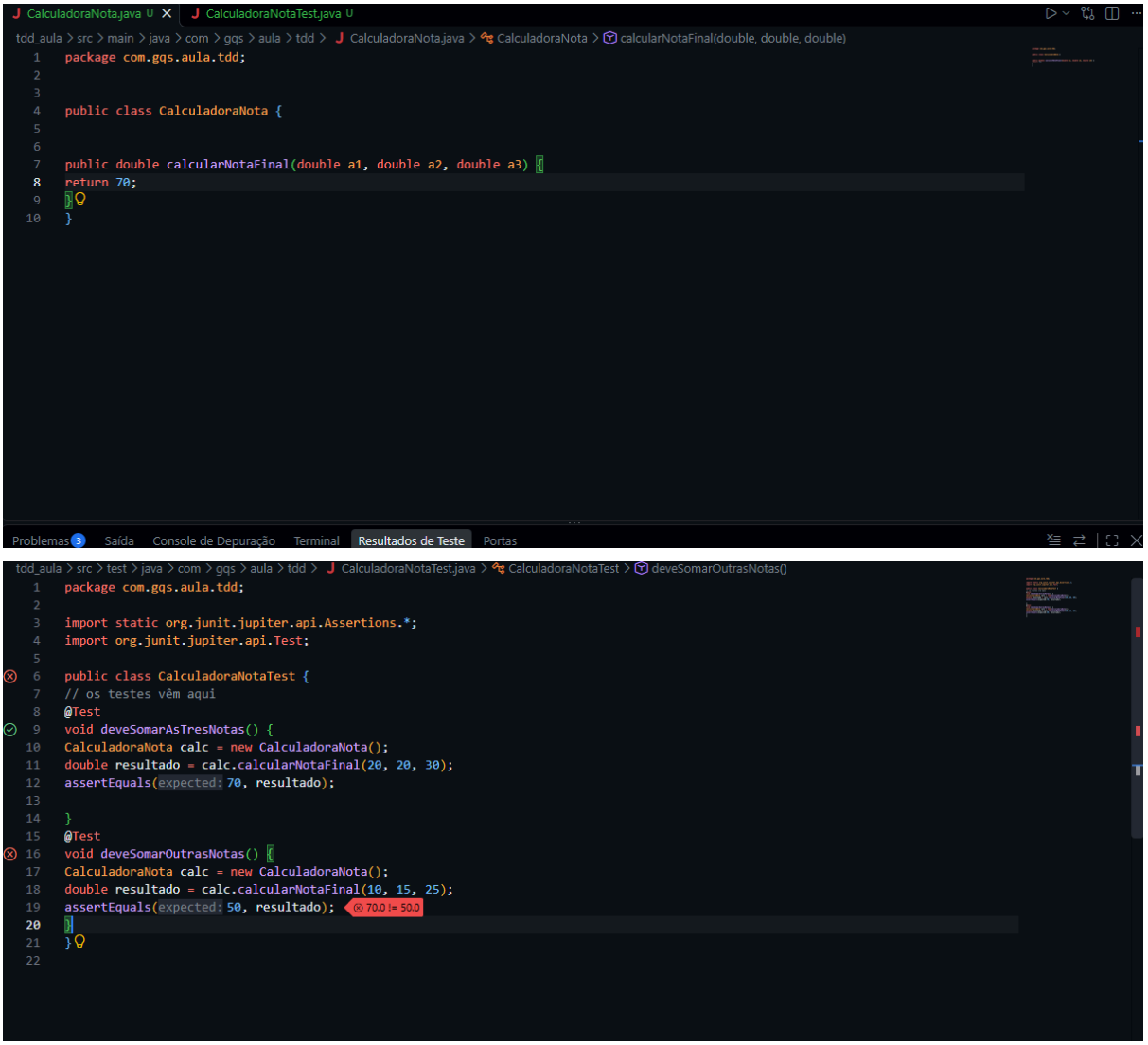
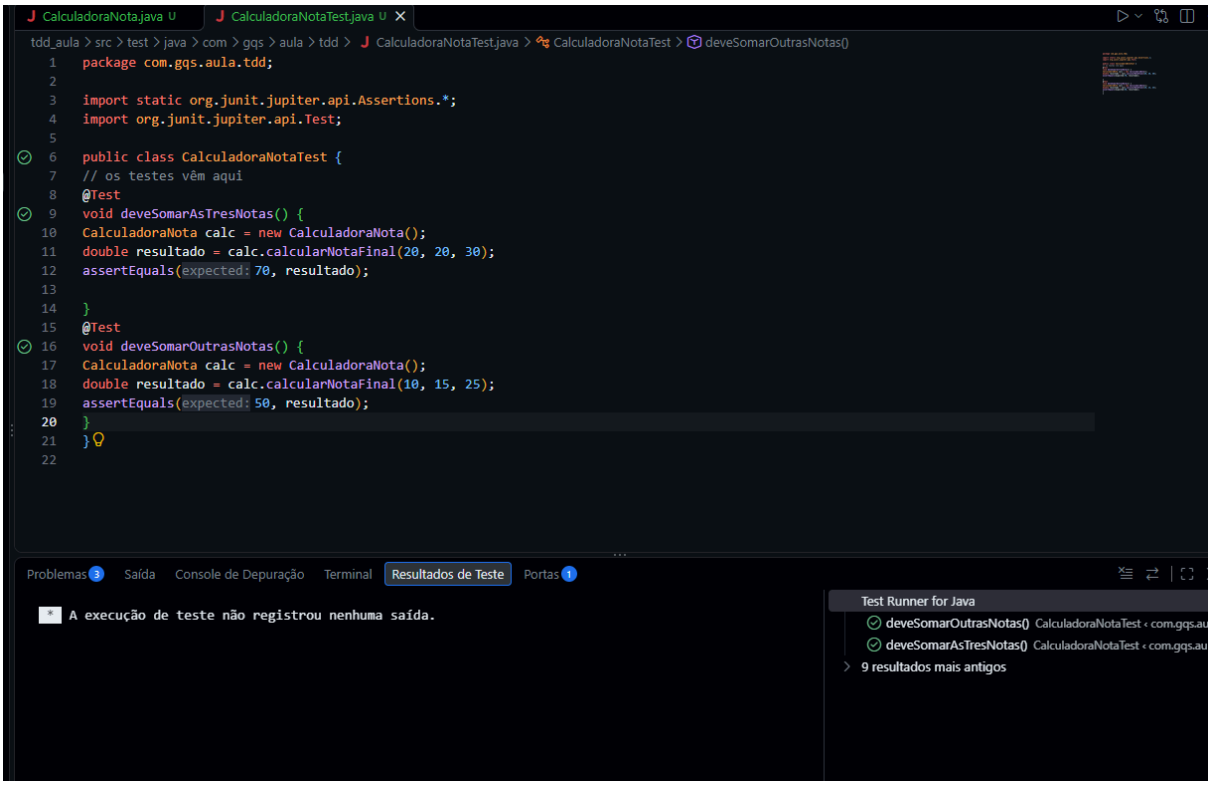


expected: <50.0> but was: <70.0>

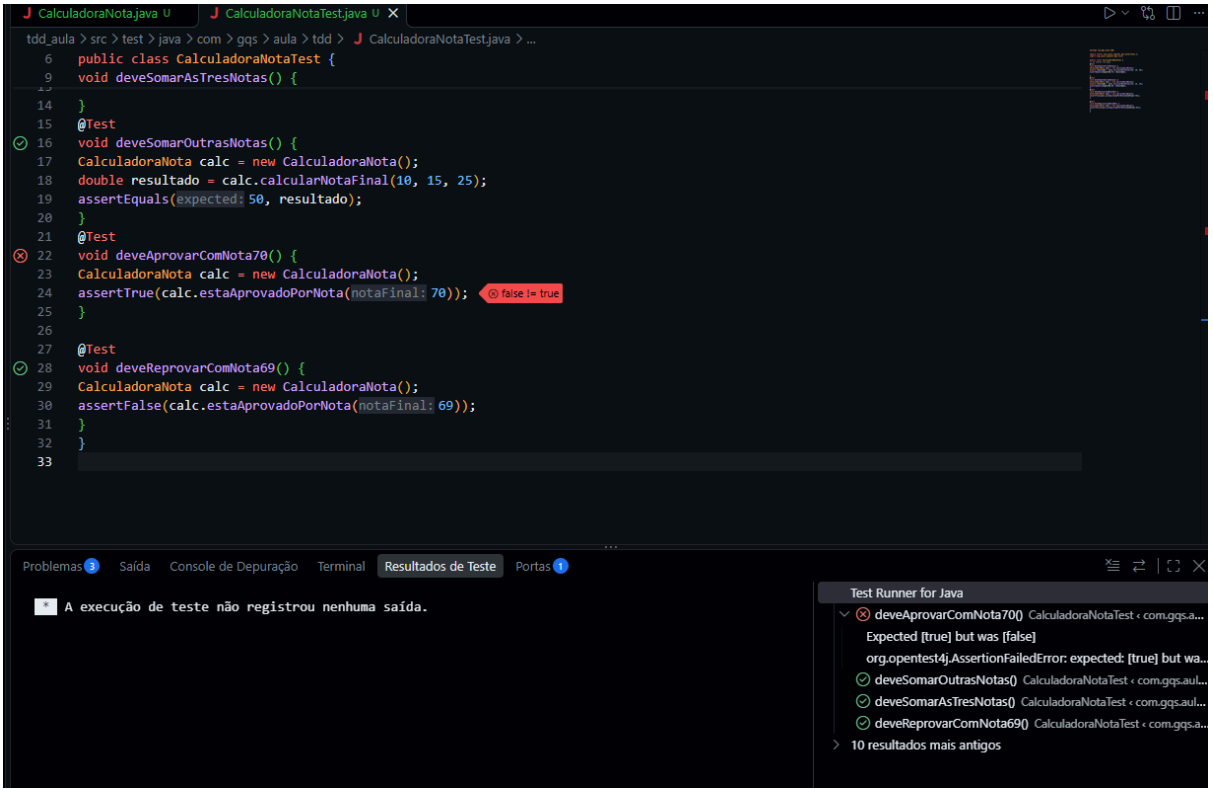


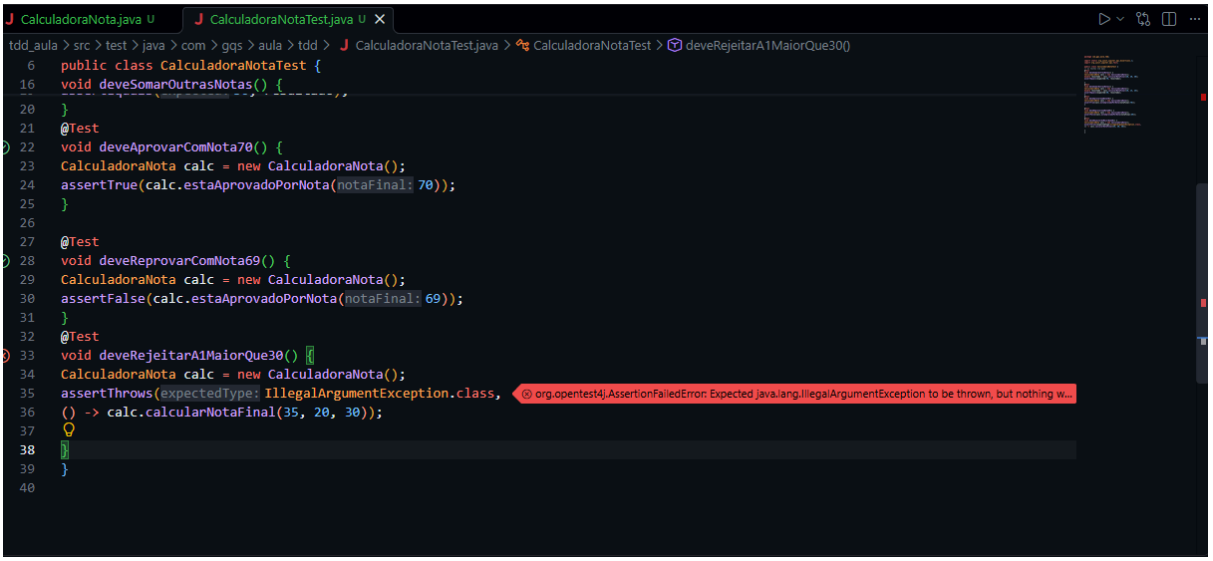

CalculadoraNota.java U
CalculadoraNotaTest.java U
tdd_aula > src > main > java > com > gqs > aula > tdd > CalculadoraNota.java > CalculadoraNota > calcularNotaFinal(double, double, double)
1 package com.gqs.aula.tdd;
2
3
4 public class CalculadoraNota {
5
6
7 public double calcularNotaFinal(double a1, double a2, double a3) {
8 return 70;
9 }
10 }
Problemas 3
Saída
Console de Depuração
Terminal
Resultados de Teste
Portas
tdd_aula > src > test > java > com > gqs > aula > tdd > CalculadoraNotaTest.java > CalculadoraNotaTest > deveSomarOutrasNotas()
1 package com.gqs.aula.tdd;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.Test;
5
6 public class CalculadoraNotaTest {
7 // os testes vêm aqui
8 @Test
9 void deveSomarAsTresNotas() {
10 CalculadoraNota calc = new CalculadoraNota();
11 double resultado = calc.calcularNotaFinal(20, 20, 30);
12 assertEquals(expected: 70, resultado);
13
14 }
15 @Test
16 void deveSomarOutrasNotas() {
17 CalculadoraNota calc = new CalculadoraNota();
18 double resultado = calc.calcularNotaFinal(10, 15, 25);
19 assertEquals(expected: 50, resultado); 70.0 != 50.0
20 }
21 }
22

return a1 + a2 +a3



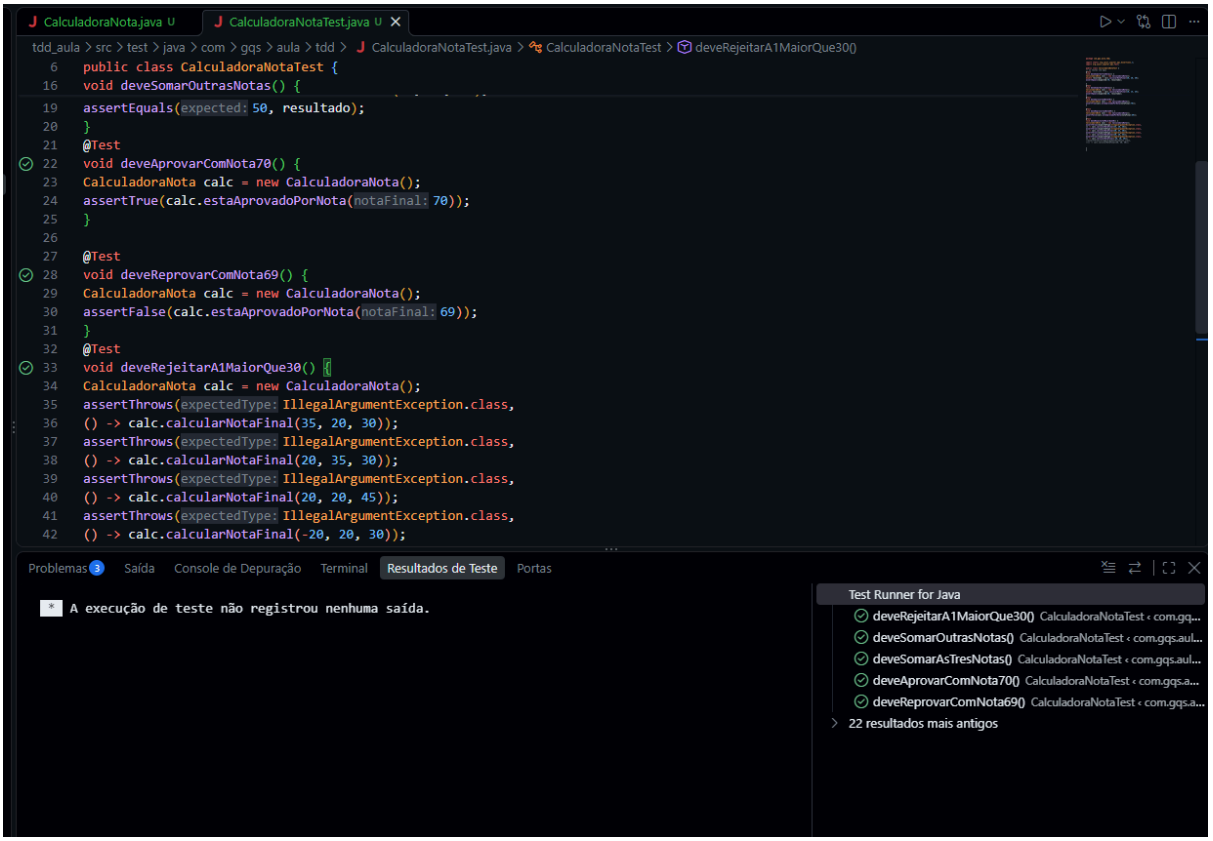
Teste na fronteira: 70 e 69




CalculadoraNota.java U
CalculadoraNotaTest.java U
tdd_aula > src > test > java > com > gqs > aula > tdd > CalculadoraNotaTest.java > CalculadoraNotaTest > deveRejeitarA1MaiorQue30()
public class CalculadoraNotaTest {
void deveSomarOutrasNotas() {
}
@Test
void deveAprovarComNota70() {
CalculadoraNota calc = new CalculadoraNota();
assertTrue(calc.estaAprovadoPorNota(notaFinal: 70));
}
@Test
void deveReprovarComNota69() {
CalculadoraNota calc = new CalculadoraNota();
assertFalse(calc.estaAprovadoPorNota(notaFinal: 69));
}
@Test
void deveRejeitarA1MaiorQue30() {
CalculadoraNota calc = new CalculadoraNota();
assertThrows(expectedType: IllegalArgumentException.class,
org.opentest4j.AssertionFailedError: Expected java.lang.IllegalArgumentException to be thrown, but nothing w...
() -> calc.calcularNotaFinal(35, 20, 30));
}


public class CalculadoraNotaTest {
@Test
void deveReprovarComNota69() {
CalculadoraNota calc = new CalculadoraNota();
assertFalse(calc.estaAprovadoPorNota(notaFinal: 69));
}
@Test
void deveRejeitarA1MaiorQue30() {
CalculadoraNota calc = new CalculadoraNota();
assertThrows(expectedType: IllegalArgumentException.class,
() -> calc.calcularNotaFinal(35, 20, 30));
assertThrows(expectedType: IllegalArgumentException.class,
() -> calc.calcularNotaFinal(20, 35, 30));
assertThrows(expectedType: IllegalArgumentException.class,
() -> calc.calcularNotaFinal(20, 20, 45));
assertThrows(expectedType: IllegalArgumentException.class,
() -> calc.calcularNotaFinal(-20, 20, 30));
assertThrows(expectedType: IllegalArgumentException.class,
() -> calc.calcularNotaFinal(30, 30, 40));
}


CalculadoraNota.java U
CalculadoraNotaTest.java U
tdd_aula > src > main > java > com > gqs > aula > tdd > CalculadoraNota.java > CalculadoraNota > calcularNotaFinal(double, double, double)
public class CalculadoraNota {
public double calcularNotaFinal(double a1, double a2, double a3) {
if (a1 < 0 || a1 > 30) {
throw new IllegalArgumentException(s: "A1 inválida");
}
if (a2 < 0 || a2 > 30) {
throw new IllegalArgumentException(s: "A2 inválida");
}
if (a3 < 0 || a3 > 40) {
throw new IllegalArgumentException(s: "A3 inválida");
}
return a1 + a2 + a3;
}
public static final double NOTA_MINIMA_APROVACAO = 70;
public boolean estaAprovadoPorNota(double notaFinal) {
return notaFinal >= NOTA_MINIMA_APROVACAO;
}



Três if parecidos viram um método



Nota e frequência decidem a situação

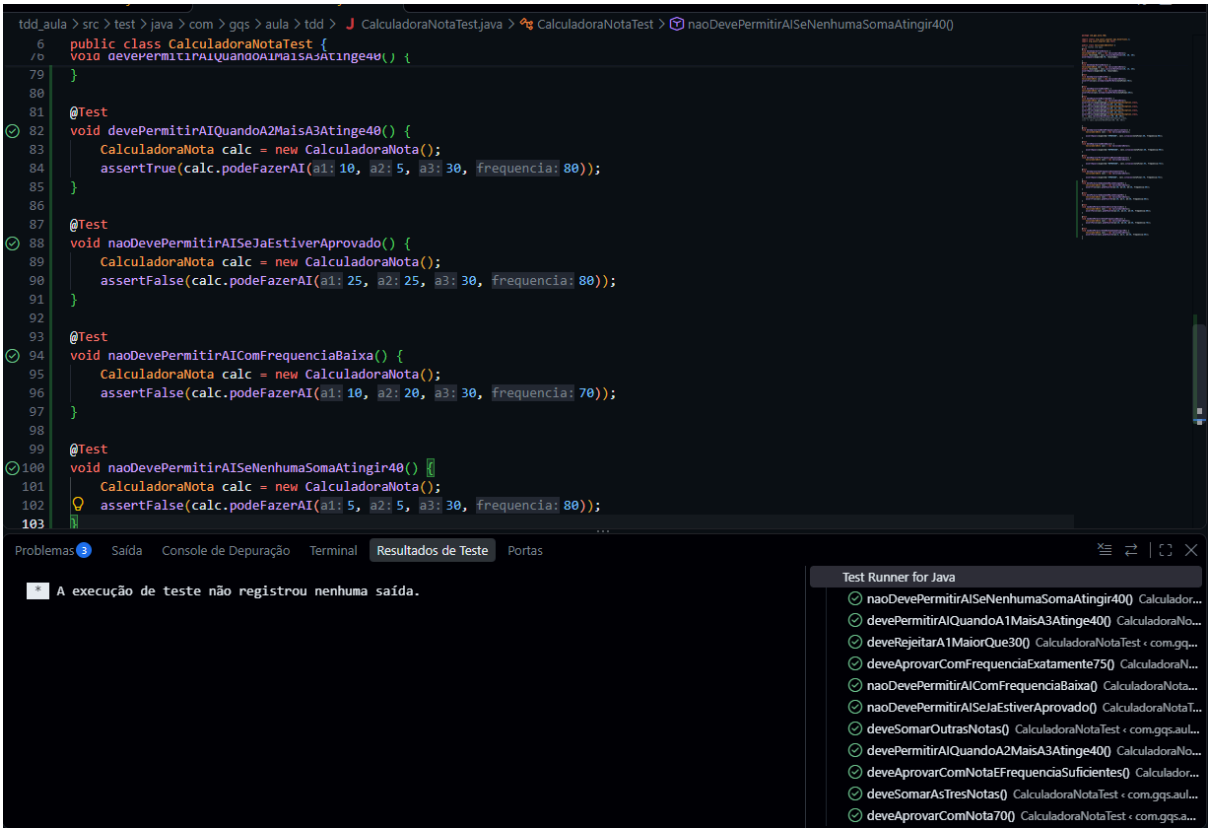


```
CalculadoraNota.java U   CalculadoraNotaTest.java U
tdd_aula > src > test > java > com > gqs > aula > tdd > CalculadoraNotaTest.java > CalculadoraNotaTest > deveAprovarComFrequenciaExatamente75()
 6  public class CalculadoraNotaTest {
47  }
48  @Test
49  void deveAprovarComNotaEFrequenciaSuficientes() {
50      CalculadoraNota calc = new CalculadoraNota();
51
52      assertEquals(expected: "APROVADO", calc.situacao(notaFinal: 80, frequencia: 90));
53  }
54
55  @Test
56  void deveReprovarComNotaBaixa() {
57      CalculadoraNota calc = new CalculadoraNota();
58
59      assertEquals(expected: "REPROVADO", calc.situacao(notaFinal: 60, frequencia: 90));
60  }
61
62  @Test
63  void deveReprovarPorFrequenciaMesmoComNotaAlta() {
64      CalculadoraNota calc = new CalculadoraNota();
65
66      assertEquals(expected: "REPROVADO", calc.situacao(notaFinal: 95, frequencia: 74));
67  }
68
69  @Test
70  void deveAprovarComFrequenciaExatamente75() {
71      CalculadoraNota calc = new CalculadoraNota();
```



```
tdd_aula > src > main > java > com > gqs > aula > tdd > CalculadoraNota.java > CalculadoraNota > FREQUENCIA_MINIMA
 1  package com.gqs.aula.tdd;
 2
 3  public class CalculadoraNota {
 4
 5  public static final double NOTA_MINIMA_APROVACAO = 70;
 6  public static final double FREQUENCIA_MINIMA = 75;
 7  public boolean estaAprovadoPorNota(double notaFinal) {
 8  return notaFinal >= NOTA_MINIMA_APROVACAO;
 9
10  }
11
12  public double calcularNotaFinal(double a1, double a2, double a3) {
13  validarNota(a1, notaMaxima: 30, nomeAvaliacao: "A1");
14  validarNota(a2, notaMaxima: 30, nomeAvaliacao: "A2");
15  validarNota(a3, notaMaxima: 40, nomeAvaliacao: "A3");
16  return a1 + a2 + a3;
17  }
18
19  private void validarNota(double nota, double notaMaxima, String nomeAvaliacao) {
20  if (nota < 0 || nota > notaMaxima) {
21  throw new IllegalArgumentException(
22  nomeAvaliacao + " deve estar entre 0 e " + notaMaxima);
23  }
24  }
25  public String situacao(double notaFinal, double frequencia) {
26      if (estaAprovadoPorNota(notaFinal) && frequencia >= 75) {
```

Ciclo 7: aplicar a nota da AI



aplicar a nota da AI

